RAJALAKSHMI ENGINEERING COLLEGE

Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

(An Autonomous Institution, Affiliated to Anna University, Chennai)

Department of Artificial Intelligence

V Semester B. Tech AIDS

AD23532 - Principles of Data Science

Academic Year: 2026-2027 **Batch:** 2024-2028

Ex 2: Data Discovery and Preparation

Roll Number: 241801187	**Date:** 06-10-2026
Dataset: Telco Customer Churn (IBM)	**Tools:** Python, Pandas, NumPy, Matplotlib, Seaborn, scikit-learn

Learning Objective

To perform data discovery and exploratory data analysis on a real-world dataset.

Question

A telecommunications company has observed an increase in customer churn, leading to revenue loss. The management wants to understand customer behavior and identify patterns before developing a machine learning model to predict churn. As a Data Analyst, the responsibility is to discover meaningful insights from the dataset, identify data quality issues, perform exploratory data analysis, detect outliers, and prepare the dataset for predictive analytics.

Task 1: Reuse Ex 1 Steps 1-6 and 10

Steps 1-6 and 10 of Ex 1 (import, load, explore, clean, encode, standardise, correlation heatmap) are reused. The cleaned data in original units is kept in **df_clean** and the encoded, standardised data in **df_encoded**.

Steps 1-3: Import, load and explore

In [1]:

```
# Step 1: libraries
import numpy as np, pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import LabelEncoder, OrdinalEncoder, StandardScaler
import warnings
warnings.filterwarnings('ignore')
sns.set_theme(style='whitegrid')
pd.set_option('display.max_columns', None)

# Step 2-3: load and explore
df = pd.read_csv('Telco-Customer-Churn.csv')
print("Shape:", df.shape)
print("Duplicate rows:", df.duplicated().sum(), "| Duplicate customerID:", df['customerID'].duplicated().sum())
print("Missing values (isnull):", df.isnull().sum().sum())
```

Out [1]:

```
Shape: (7043, 21)
Duplicate rows: 0 | Duplicate customerID: 0
Missing values (isnull): 0
```

Step 4: Clean the data

In [2]:

```
# Step 4: clean
text_cols = df.select_dtypes(exclude='number').columns
df[text_cols] = df[text_cols].apply(lambda s: s.astype(str).str.strip())
df['TotalCharges'] = pd.to_numeric(df['TotalCharges'], errors='coerce')   # 11 blanks -> NaN
print("Missing after type fix:", df.isnull().sum()[lambda x: x > 0].to_dict())

df = df.drop_duplicates().drop_duplicates(subset='customerID', keep='first')

for col in df.select_dtypes(include='number').columns:      # numerical -> median
    if df[col].isnull().sum() > 0:
        df[col] = df[col].fillna(df[col].median())
for col in df.select_dtypes(exclude='number').columns:      # categorical -> mode
    if df[col].isnull().sum() > 0:
        df[col] = df[col].fillna(df[col].mode()[0])

print("Total missing values remaining:", df.isnull().sum().sum())
df_clean = df.copy()
print("Cleaned dataset shape:", df_clean.shape)
```

Out [2]:

```
Missing after type fix: {'TotalCharges': 11}
Total missing values remaining: 0
Cleaned dataset shape: (7043, 21)
```

Steps 5-6: Encode categorical columns and standardise numerical features

In [3]:

```
# Step 5: encode
df_encoded = df_clean.copy()
binary_cols = ['gender', 'Partner', 'Dependents', 'Churn']
le = LabelEncoder()
for col in binary_cols:
    df_encoded[col] = le.fit_transform(df_encoded[col])
df_encoded['Contract'] = OrdinalEncoder(
    categories=[['Month-to-month', 'One year', 'Two year']]).fit_transform(df_encoded[['Contract']]).astype(int)
df_encoded = pd.get_dummies(df_encoded, columns=['PaymentMethod', 'InternetService'], dtype=int)
print("Shape after encoding:", df_encoded.shape)

# Step 6: standardise
scale_cols = ['tenure', 'MonthlyCharges', 'TotalCharges']
df_encoded[scale_cols] = StandardScaler().fit_transform(df_encoded[scale_cols])
display(df_encoded[scale_cols].describe().round(2))
```

Out [3]:

```
Shape after encoding: (7043, 26)
```

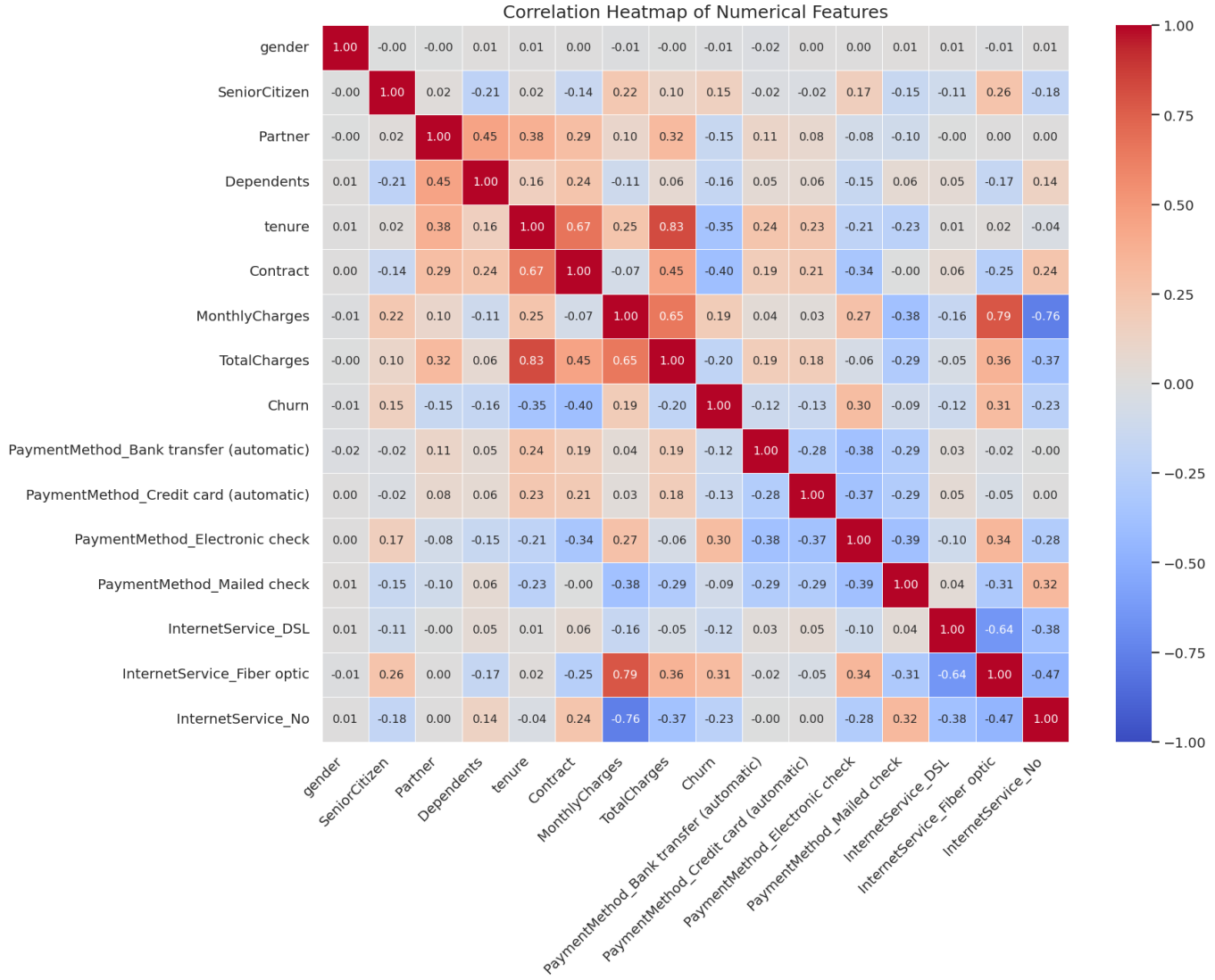
	tenure	MonthlyCharges	TotalCharges
count	7043.0	7043.0	7043.0
mean	-0.0	-0.0	-0.0
std	1.0	1.0	1.0
min	-1.32	-1.55	-1.0
25%	-0.95	-0.97	-0.83
50%	-0.14	0.19	-0.39
75%	0.92	0.83	0.66
max	1.61	1.79	2.83

Step 10: Correlation heatmap

In [4]:

```
# Step 10: correlation heatmap
corr = df_encoded.select_dtypes(include='number').corr()
plt.figure(figsize=(14, 11))
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm', vmin=-1, vmax=1, center=0,
            linewidths=0.5, annot_kws={'size': 9})
plt.title('Correlation Heatmap of Numerical Features', fontsize=14)
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```

Out [4]:



Task 2: Distribution of Churn, Gender, Contract Type and Payment Method

In [5]:

```
churn_palette = {'No': '#4C72B0', 'Yes': '#DD8452'}
fig, axes = plt.subplots(2, 2, figsize=(13, 9))
for ax, col in zip(axes.ravel(), ['Churn', 'gender', 'Contract', 'PaymentMethod']):
    order = df_clean[col].value_counts().index
    sns.countplot(data=df_clean, x=col, order=order, color='steelblue', ax=ax)
    for cont in ax.containers:
        ax.bar_label(cont, fontsize=9)
    ax.set_title(f'Distribution of {col}')
    ax.set_ylabel('Number of Customers')
    if col == 'PaymentMethod':
        ax.tick_params(axis='x', labelsize=8)
plt.tight_layout()
plt.show()

for col in ['Churn', 'gender', 'Contract', 'PaymentMethod']:
    print(f"\n{col} (%):")
    print((df_clean[col].value_counts(normalize=True) * 100).round(2).to_string())
```

Out [5]:

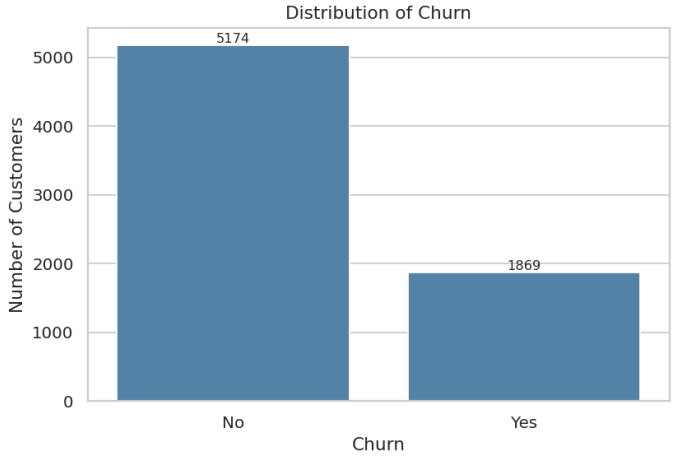
```
Churn (%):
Churn
No     73.46
Yes    26.54

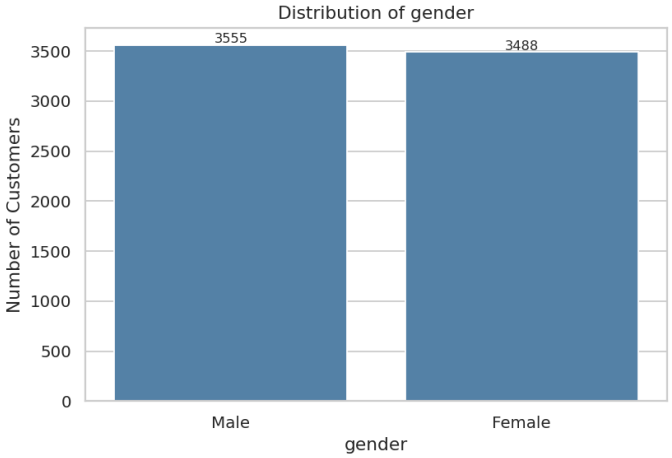
gender (%):
gender
Male      50.48
Female    49.52

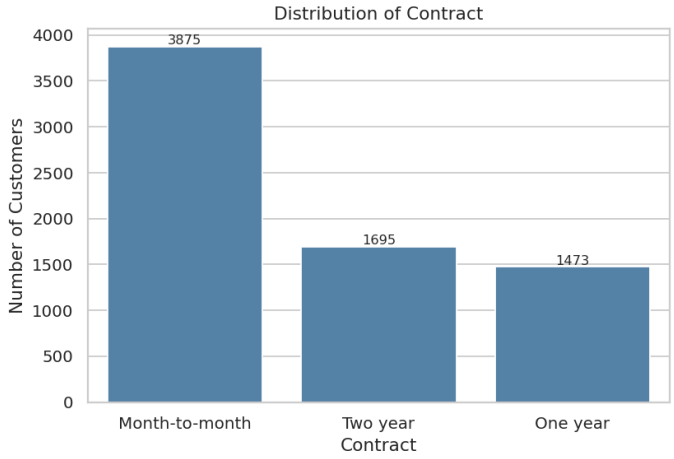
Contract (%):
Contract
Month-to-month    55.02
```

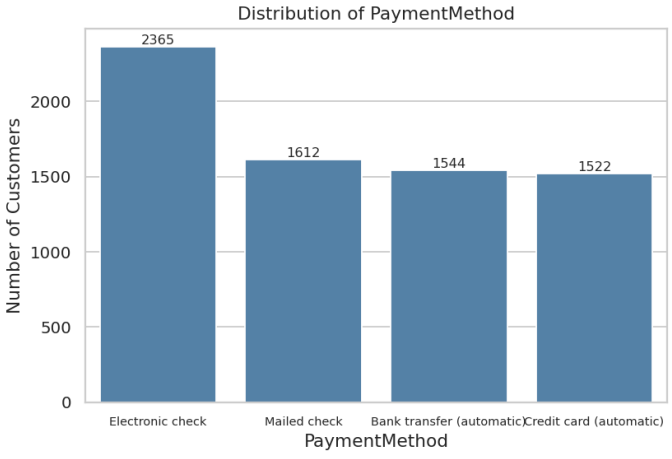
```
Two year          24.07
One year          20.91

PaymentMethod (%):
PaymentMethod
Electronic check             33.58
Mailed check                 22.89
Bank transfer (automatic)    21.92
Credit card (automatic)      21.61
```









> **Observation:** The dataset is imbalanced: 73.46% of customers stayed and 26.54% churned. Gender is almost evenly split (50.48% male, 49.52% female). Month-to-month is the most common contract (55.02%), and electronic check is the most common payment method (33.58%).

Task 3: Histograms and Boxplots of MonthlyCharges, TotalCharges and Tenure

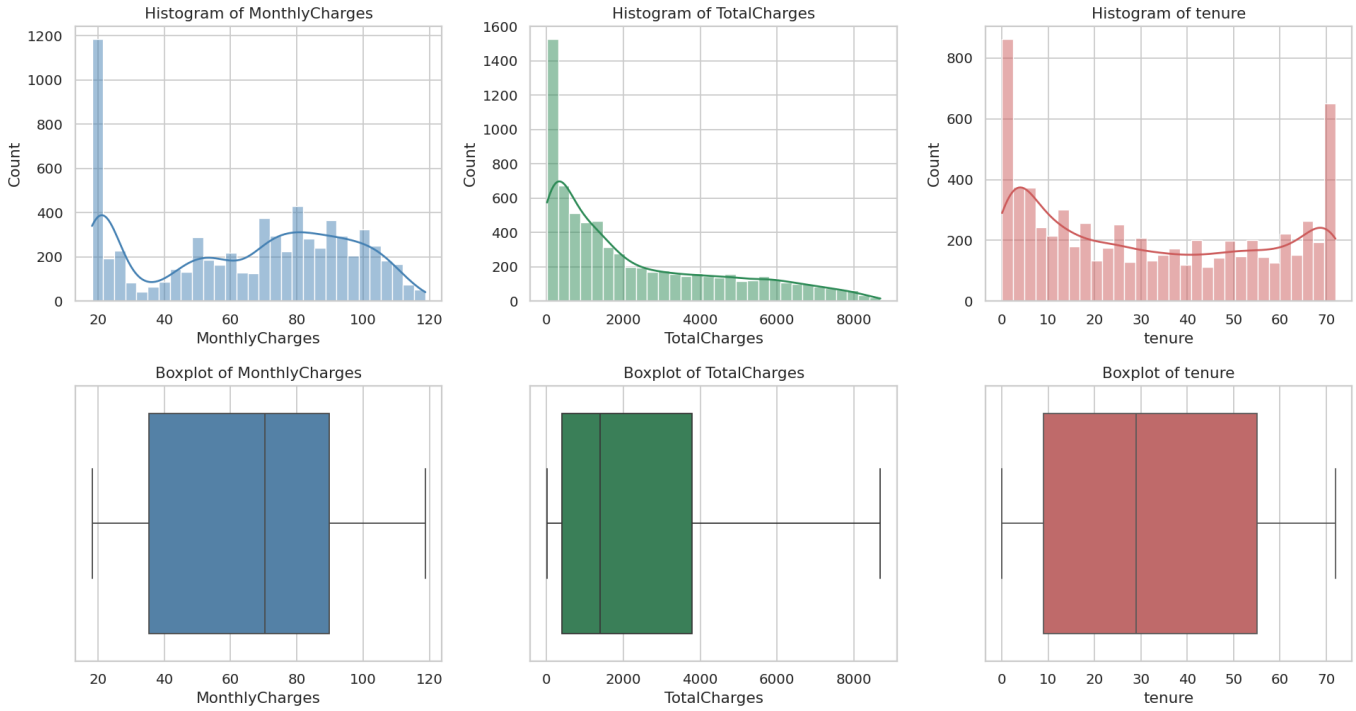
In [6]:

```
num_features = ['MonthlyCharges', 'TotalCharges', 'tenure']
colors = ['steelblue', 'seagreen', 'indianred']
fig, axes = plt.subplots(2, 3, figsize=(15, 8))
for i, (col, c) in enumerate(zip(num_features, colors)):
    sns.histplot(df_clean[col], bins=30, kde=True, color=c, ax=axes[0, i])
    axes[0, i].set_title(f'Histogram of {col}')
    sns.boxplot(x=df_clean[col], color=c, ax=axes[1, i])
    axes[1, i].set_title(f'Boxplot of {col}')
plt.tight_layout()
plt.show()

print(df_clean[num_features].agg(['mean', 'median', 'skew']).round(2))
```

Out [6]:

```
        MonthlyCharges  TotalCharges  tenure
mean             64.76       2281.92   32.37
median           70.35       1397.48   29.00
skew             -0.22          0.96    0.24
```



Observation: MonthlyCharges is bimodal (mean 64.76 below median 70.35). TotalCharges is right-skewed (skewness 0.96, mean 2281.92 well above median 1397.48) because it accumulates with tenure. Tenure is U-shaped, with peaks for very new and very long-standing customers. None of the boxplots show points beyond the whiskers.

Task 4: Detect outliers using the Interquartile Range (IQR) method

In [7]:

```
rows = []
for col in ['tenure', 'MonthlyCharges', 'TotalCharges']:
    q1, q3 = df_clean[col].quantile([0.25, 0.75])
    iqr = q3 - q1
    low, high = q1 - 1.5 * iqr, q3 + 1.5 * iqr
    n_out = ((df_clean[col] < low) | (df_clean[col] > high)).sum()
    rows.append([col, q1, q3, iqr, low, high, n_out, round(n_out / len(df_clean) * 100, 2)])

outlier_table = pd.DataFrame(rows, columns=['Feature', 'Q1', 'Q3', 'IQR', 'Lower bound',
                                            'Upper bound', 'Outliers', 'Outliers (%)']).round(2)
display(outlier_table.set_index('Feature'))
```

Out [7]:

Feature	Q1	Q3	IQR	Lower bound	Upper bound	Outliers	Outliers (%)
tenure	9.0	55.0	46.0	-60.0	124.0	0	0.0
MonthlyCharges	35.5	89.85	54.35	-46.02	171.38	0	0.0
TotalCharges	402.22	3786.6	3384.38	-4674.34	8863.16	0	0.0

Observation: With the 1.5 x IQR rule, no values fall outside the lower and upper bounds in tenure, MonthlyCharges or TotalCharges (0 outliers, 0%). The right skew of TotalCharges is natural variation, so no rows needed to be removed or capped.

Task 5: Relationship of features with Churn

In [8]:

```
fig, axes = plt.subplots(2, 2, figsize=(13, 9))

sns.countplot(data=df_clean, x='Contract', hue='Churn', palette=churn_palette,
              order=['Month-to-month', 'One year', 'Two year'], hue_order=['No', 'Yes'], ax=axes[0, 0])
axes[0, 0].set_title('Contract Type vs Churn')

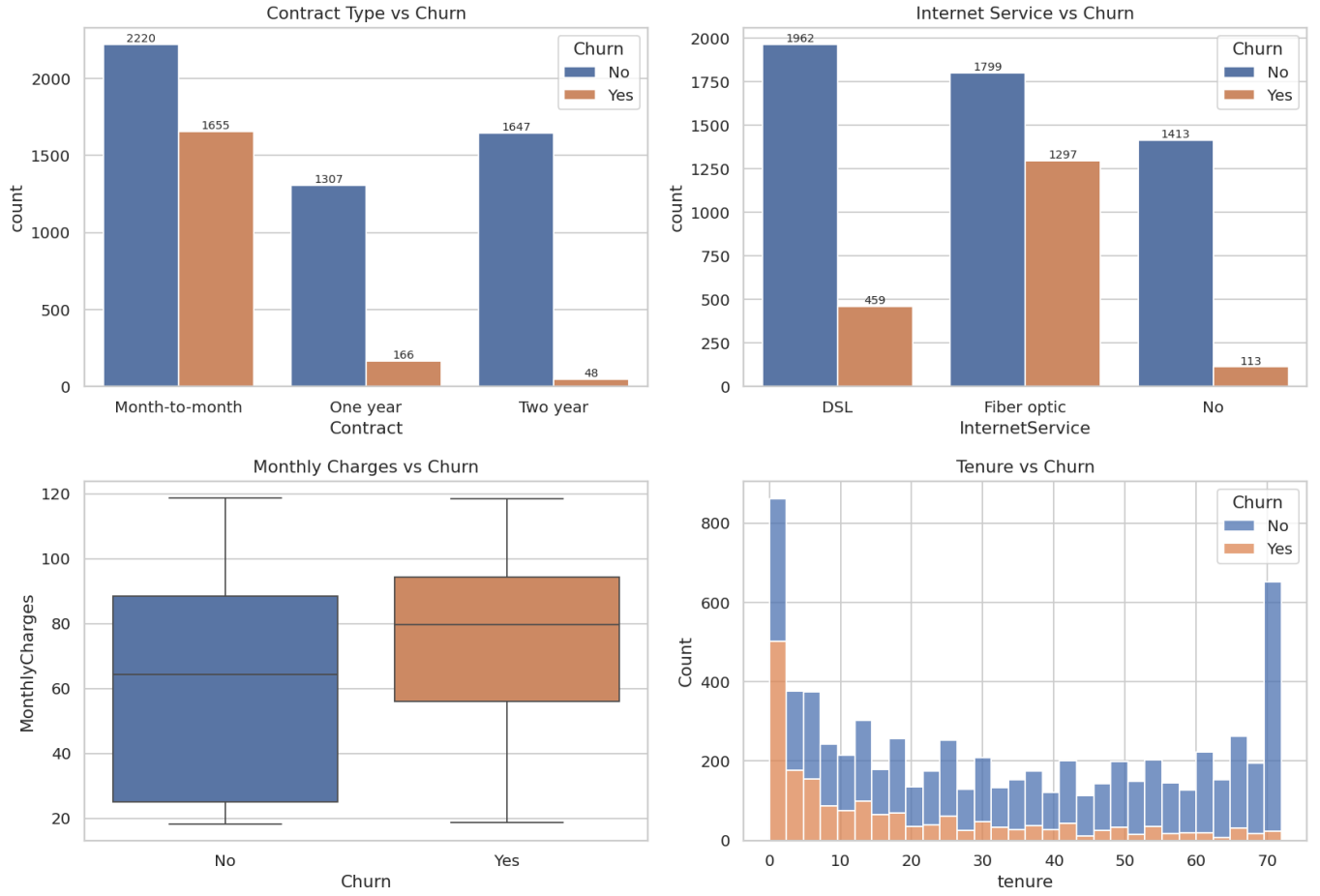
sns.countplot(data=df_clean, x='InternetService', hue='Churn', palette=churn_palette,
              order=['DSL', 'Fiber optic', 'No'], hue_order=['No', 'Yes'], ax=axes[0, 1])
axes[0, 1].set_title('Internet Service vs Churn')

sns.boxplot(data=df_clean, x='Churn', y='MonthlyCharges', hue='Churn', palette=churn_palette,
            order=['No', 'Yes'], legend=False, ax=axes[1, 0])
axes[1, 0].set_title('Monthly Charges vs Churn')

sns.histplot(data=df_clean, x='tenure', hue='Churn', palette=churn_palette, bins=30,
             multiple='stack', ax=axes[1, 1])
axes[1, 1].set_title('Tenure vs Churn')

for ax in axes[0]:
    for cont in ax.containers:
        ax.bar_label(cont, fontsize=8)
plt.tight_layout()
plt.show()
```

Out [8]:



Observation: Churn is concentrated among month-to-month contracts, fiber optic internet users, customers with higher monthly charges, and customers with short tenure. Customers on two-year contracts, without internet service, or with long tenure rarely churn.

Task 6: Analytical questions

Questions 1-3

In [9]:

```
churned = (df_clean['Churn'] == 'Yes').astype(int)
print("Q1. Churn rate (%) by contract type:")
q1 = (churned.groupby(df_clean['Contract']).mean() * 100).round(2).sort_values(ascending=False)
print(q1.to_string())
print("-> Highest churn:", q1.idxmax())

print("\nQ2. Payment methods of customers who churned:")
q2 = df_clean.loc[churned == 1, 'PaymentMethod'].value_counts()
print(q2.to_string())
print("-> Most frequent:", q2.idxmax())

print("\nQ3. Churn rate (%) by tenure group:")
bins = pd.cut(df_clean['tenure'], [-1, 12, 24, 48, 72], labels=['0-12', '13-24', '25-48', '49-72'])
print((churned.groupby(bins).mean() * 100).round(2).to_string())
print("Mean tenure (months) - churned:", round(df_clean.loc[churned == 1, 'tenure'].mean(), 1),
      "| retained:", round(df_clean.loc[churned == 0, 'tenure'].mean(), 1))
```

Out [9]:

```
Q1. Churn rate (%) by contract type:
Contract
Month-to-month    42.71
One year          11.27
Two year           2.83
-> Highest churn: Month-to-month

Q2. Payment methods of customers who churned:
PaymentMethod
Electronic check             1071
Mailed check                  308
Bank transfer (automatic)     258
Credit card (automatic)       232
-> Most frequent: Electronic check

Q3. Churn rate (%) by tenure group:
tenure
0-12     47.44
13-24    28.71
25-48    20.39
49-72     9.51
Mean tenure (months) - churned: 18.0 | retained: 37.6
```

> **Observation:** Month-to-month contracts have the highest churn (42.71%). Electronic check is the most common payment method among churners (1071 of 1869). Churn falls steadily as tenure grows, from 47.44% in the first year to 9.51% after four years.

Questions 4-7

In [10]:

```
print("Q4. Average MonthlyCharges ($) by customer group:")
groups = {}
for col in ['InternetService', 'Contract', 'gender', 'SeniorCitizen', 'Partner', 'Dependents',
            'PaymentMethod', 'Churn']:
    for k, v in df_clean.groupby(col)['MonthlyCharges'].mean().items():
        groups[f"{col} = {k}"] = v
g = pd.Series(groups).sort_values(ascending=False).round(2)
print(g.head(6).to_string())
print("-> Highest:", g.index[0])

print("\nQ5. Churn rate (%) by Internet Service:")
q5 = (churned.groupby(df_clean['InternetService']).mean() * 100).round(2).sort_values(ascending=False)
print(q5.to_string())
print("-> Highest churn:", q5.idxmax())

print("\nQ6. MonthlyCharges vs Churn:")
print(df_clean.groupby('Churn')['MonthlyCharges'].agg(['mean', 'median']).round(2).to_string())
print("Correlation (MonthlyCharges, Churn):", round(df_clean['MonthlyCharges'].corr(churned), 3))

print("\nQ7. Correlation with Churn (strongest first):")
cc = corr['Churn'].drop('Churn')
print(cc.reindex(cc.abs().sort_values(ascending=False).index).head(8).round(3).to_string())
```

Out [10]:

```
Q4. Average MonthlyCharges ($) by customer group:
InternetService = Fiber optic               91.50
SeniorCitizen = 1                           79.82
PaymentMethod = Electronic check            76.26
Churn = Yes                                 74.44
Partner = Yes                               67.78
PaymentMethod = Bank transfer (automatic)   67.19
-> Highest: InternetService = Fiber optic

Q5. Churn rate (%) by Internet Service:
```

```
InternetService
Fiber optic    41.89
DSL            18.96
No              7.40
-> Highest churn: Fiber optic

Q6. MonthlyCharges vs Churn:
        mean  median
Churn
No     61.27   64.43
Yes    74.44   79.65
Correlation (MonthlyCharges, Churn): 0.193

Q7. Correlation with Churn (strongest first):
Contract                         -0.397
tenure                           -0.352
InternetService_Fiber optic       0.308
PaymentMethod_Electronic check    0.302
InternetService_No               -0.228
TotalCharges                     -0.199
MonthlyCharges                    0.193
Dependents                       -0.164
```

> **Observation:** Fiber optic customers pay the most on average ($91.50) and also churn the most (41.89%). Churned customers pay more per month (mean $74.44, median $79.65) than retained customers (mean $61.27, median $64.43).

Answers to the analytical questions

1. **Contract type with the highest churn:** month-to-month (42.71%), versus 11.27% for one-year and 2.83% for two-year.
2. **Payment method most used by churners:** electronic check (1071 of the 1869 churned customers).
3. **Shorter tenure and churn:** yes. Customers who churn have a mean tenure of 18.0 months against 37.6 for those who stay, and churn drops from 47.44% (0-12 months) to 9.51% (49-72 months).
4. **Group with the highest monthly charges:** fiber optic internet customers (average $91.50), followed by senior citizens ($79.82).
5. **Internet service with the highest churn:** fiber optic (41.89%), versus DSL 18.96% and no internet 7.40%.
6. **Monthly charges and churn:** a weak positive relationship (correlation 0.193). Churners pay more, but price alone does not explain churn.
7. **Strongest indicators of churn:** Contract (-0.397), tenure (-0.352), fiber optic internet (0.308) and electronic check payment (0.302). Gender has almost no effect.

Task 7: Split the cleaned dataset into Training (80%) and Testing (20%)

Before splitting, the remaining text columns (PhoneService, PaperlessBilling, MultipleLines, OnlineSecurity, OnlineBackup, DeviceProtection, TechSupport, StreamingTV, StreamingMovies) are encoded so that the final dataset is fully numerical. customerID is dropped because it is only an identifier.

In [11]:

```
from sklearn.model_selection import train_test_split

df_final = df_encoded.drop(columns='customerID')
for col in ['PhoneService', 'PaperlessBilling']:                        # binary Yes/No
    df_final[col] = (df_final[col] == 'Yes').astype(int)
remaining = [c for c in df_final.columns if not pd.api.types.is_numeric_dtype(df_final[c])]
df_final = pd.get_dummies(df_final, columns=remaining, dtype=int)      # 3-category columns
print("Non-numeric columns left:", [c for c in df_final.columns if not pd.api.types.is_numeric_dtype(df_final[c])])
print("Final dataset shape:", df_final.shape)

X = df_final.drop(columns='Churn')
y = df_final['Churn']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=42, stratify=y)

print("\nX_train:", X_train.shape, "| X_test:", X_test.shape)
print("y_train:", y_train.shape, "| y_test:", y_test.shape)
print("Training share: %.1f%% | Testing share: %.1f%%" % (len(X_train)/len(X)*100, len(X_test)/len(X)*100))
print("Churn rate - train: %.2f%% | test: %.2f%%" % (y_train.mean()*100, y_test.mean()*100))
```

Out [11]:

```
Non-numeric columns left: []
Final dataset shape: (7043, 39)

X_train: (5634, 38) | X_test: (1409, 38)
y_train: (5634,) | y_test: (1409,)
Training share: 80.0% | Testing share: 20.0%
Churn rate - train: 26.54% | test: 26.54%
```

> **Observation:** The data was split 80:20 into 5,634 training and 1,409 testing rows with random_state=42. Stratifying on Churn keeps the churn rate at 26.54% in both sets.

Task 8: Save the final processed dataset as CustomerChurn_Preprocessed.csv

In [12]:

```
df_final.to_csv('CustomerChurn_Preprocessed.csv', index=False)

check = pd.read_csv('CustomerChurn_Preprocessed.csv')
print("File saved: CustomerChurn_Preprocessed.csv")
print("Rows x columns:", check.shape)
print("Missing values:", check.isnull().sum().sum())
display(check.head(5))
```

Out [12]:

```
File saved: CustomerChurn_Preprocessed.csv
Rows x columns: (7043, 39)
Missing values: 0
```

	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	Contract	PaperlessBilling	MonthlyCharges
0.0	0.0	0.0	1.0	0.0	-1.2774445836787656	0.0	0.0	1.0	-1.160322916034919
1.0	1.0	0.0	0.0	0.0	0.0663274190822359	1.0	1.0	0.0	-0.2596289419448806
2.0	1.0	0.0	0.0	0.0	-1.2367242199587352	1.0	0.0	1.0	-0.3626603559551803
3.0	1.0	0.0	0.0	0.0	0.5142514200025698	0.0	1.0	0.0	-0.7465354629935549
4.0	0.0	0.0	0.0	0.0	-1.2367242199587352	1.0	0.0	1.0	0.1973652331008032

	TotalCharges	Churn	PaymentMethod_Bank transfer (automatic)	PaymentMethod_Credit card (automatic)	PaymentMethod_Electronic check
0.0	-0.9942419353101232	0.0	0.0	0.0	1.0
1.0	-0.1732441257495937	0.0	0.0	0.0	0.0
2.0	-0.959674071202044	1.0	0.0	0.0	0.0
3.0	-0.1947662633647771	0.0	1.0	0.0	0.0
4.0	-0.9404697022531112	1.0	0.0	0.0	1.0

	PaymentMethod_Mailed check	InternetService_DSL	InternetService_Fiber optic	InternetService_No	MultipleLines_No
0	0	1	0	0	0
1	1	1	0	0	1
2	1	1	0	0	1
3	0	1	0	0	0
4	0	0	1	0	1

	MultipleLines_No phone service	MultipleLines_Yes	OnlineSecurity_No	OnlineSecurity_No internet service	OnlineSecurity_Yes
0	1	0	1	0	0
1	0	0	0	0	1
2	0	0	0	0	1
3	1	0	0	0	1
4	0	0	1	0	0

	OnlineBackup_No	OnlineBackup_No internet service	OnlineBackup_Yes	DeviceProtection_No	DeviceProtection_No internet service
0	0	0	1	1	0
1	1	0	0	0	0
2	0	0	1	1	0
3	1	0	0	0	0
4	1	0	0	1	0

	DeviceProtection_Yes	TechSupport_No	TechSupport_No internet service	TechSupport_Yes	StreamingTV_No
0	0	1	0	0	1
1	1	1	0	0	1
2	0	1	0	0	1
3	1	0	0	1	1
4	0	1	0	0	1

	StreamingTV_No internet service	StreamingTV_Yes	StreamingMovies_No	StreamingMovies_No internet service	StreamingMovies_Yes
0	0	0	1	0	0
1	0	0	1	0	0
2	0	0	1	0	0
3	0	0	1	0	0
4	0	0	1	0	0

> **Observation:** The final dataset (7,043 rows, 39 columns, all numerical, no missing values) was saved and read back successfully to confirm it.

Result

Thus data discovery and exploratory data analysis were performed on the Telco Customer Churn dataset. The distributions of the key variables were studied, no outliers were found using the IQR method, and the relationships of contract type, internet service, monthly charges and tenure with churn were visualised. Contract type, tenure, fiber optic internet and electronic check payment were identified as the strongest indicators of churn. The cleaned dataset was split into training (80%) and testing (20%) sets and saved as CustomerChurn_Preprocessed.csv.